

started to evolve. Also, HDFS was widely accepted for Big Data storage. It did not make sense to replicate data for other frameworks. The Hadoop community worked on re-hauling the platform to take it beyond MapReduce. The result of this was Hadoop 2.0, which separated resource management from application management. The resource management system was named Yarn.

Yarn is again a master-slave architecture, with the resource manager acting as a master that manages the resource assignments to the different applications on the cluster. The slave component is called the NodeManager, runs on every node in the cluster, and is responsible for launching the compute containers needed by the application.

The ApplicationMaster is the framework-specific entity. It is responsible for negotiating resources from the ResourceManager and working with the node manager to submit and monitor the application tasks.

This decoupling allowed other frameworks to work alongside MapReduce, accessing and sharing data on the same cluster, thereby helping to improve cluster utilisation.

Apache Mesos (<http://mesos.apache.org/>) is a generic cluster resource management framework that can manage every resource in the data centre. Mesos differs from Yarn in the way the scheduling works. Mesos implements a two-level scheduling mechanism, where the master makes resource offers to the framework schedulers, and the frameworks decide whether to accept or decline them. This model enables Mesos to become very scalable and generic, and allows frameworks to meet specific goals such as data locality really well.

Mesos is a master/slave architecture with the Mesos master running on one of the nodes, and is shadowed by several standby masters that can takeover in case of a failure. The master manages the slave processes on the cluster nodes and the frameworks that run tasks on the nodes. The framework running on Mesos has two components: a scheduler that registers with the master, and the framework executor that launches on the Mesos slave nodes. In Mesos, the slave nodes report to the master about the available resources on offer. The Mesos master looks up the allocation policies and offers the resources to the framework as per the policy. The framework, based on its goal and the tasks that need to be run, accepts the offer completely, partially or can even decline it. It sends back a response with the acceptance and the tasks to be run, if any. The Mesos master forwards the tasks to the corresponding slaves, which allocate the offered resources to the executor, and the executor in turn launches the tasks.

Machine learning libraries

Big Data would not be worth the effort if it didn't

provide a business value at the end. Machine learning programs the systems to learn or process large amounts of data, and be able to apply the learnings to predict outcomes on an unseen input dataset. Machine learning systems have enabled several real world use cases such as targeted ad campaigns, recommendation engines, 'next best offer/action' scenarios, self-learning autonomous systems, etc. We will look at a few of the frameworks in this space.

Apache Mahout (<http://mahout.apache.org/>) aims to provide a scalable machine learning platform with the implementation of several algorithms out-of-the-box, and provides a framework for implementing custom algorithms as well. Although, Apache Mahout was one of the earliest ML libraries, it was originally written for the MapReduce programming paradigm. However, MapReduce was not very well suited for the iterative nature of machine learning algorithms and hence did not find great success. However, after Spark started gaining momentum, Mahout has been ported to Apache Spark, rebranded as Spark MLlib, and has been discontinued on Hadoop MapReduce.

Spark MLlib (<https://spark.apache.org/mllib/>) is a scalable machine learning platform, which is written on top of Spark and is available as an extension of the Spark Core execution engine. Spark MLlib has an advantage as it has been implemented as a native extension to Spark Core. Spark MLlib has several algorithms written for ML problems such as classification, regression, collaborative filtering, clustering, decomposition, etc.

PredictionIO (<http://prediction.io/>) is a scalable machine learning server that provides a framework enabling faster prototyping and productionising of machine learning applications. It is built on Apache Spark and leverages Spark MLlib to provide implementation templates of several machine learning algorithms. It provides an interface to expose the trained prediction model as a service through an event server based architecture. It also provides a means to persist the trained models in a distributed environment. The events generated are collected in real-time and can be used to retrain the model as a batch job. The client application can query the service over REST APIs and get the predicted results back in a JSON response. 

By: Subhash Bylaiah and Dr B. Thangaraju

Subhash Bylaiah is a senior technologist with over 15 years of experience in database systems and application development. His current assignment is with Wipro Technologies. He can be reached at subhash.bylaiah@wipro.com.

Dr B. Thangaraju is an open source software (OSS) evangelist, who works at Talent Transformation, Wipro Technologies, Bangalore and he can be reached at balat.raju@wipro.com.